

Trabalho Prático 1: Análise Dinâmica de Carteiras de Ações

1.0

Gerado por Doxygen 1.16.1

1 Hierarquia de Diretórios	1
1.1 Diretórios	1
2 Índice dos Componentes	3
2.1 Lista de Classes	3
3 Índice dos Arquivos	5
3.1 Lista de Arquivos	5
4 Documentação do Diretório	7
4.1 Referência do diretório C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include	7
4.2 Referência do diretório C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src	7
4.3 Referência do diretório C:/Users/luigu/OneDrive/Documentos/Trabalho1ED	7
5 Classes	9
5.1 Referência da Classe Acao	9
5.1.1 Construtores e Destrutores	9
5.1.1.1 Acao()	9
5.1.1.2 ~Acao()	9
5.1.2 Documentação das funções	10
5.1.2.1 adicionarCotacao()	10
5.1.2.2 calcularAvgRet()	10
5.1.2.3 calcularCons()	10
5.1.2.4 calcularRet()	11
5.1.2.5 calcularRetornoElementar()	11
5.1.2.6 calcularStab()	11
5.1.2.7 getId()	11
5.1.2.8 getUltimaCotacao()	12
5.2 Referência da Classe Cliente	12
5.2.1 Construtores e Destrutores	12
5.2.1.1 Cliente()	12
5.2.1.2 ~Cliente()	12
5.2.2 Documentação das funções	13
5.2.2.1 adicionarAcao()	13
5.2.2.2 getId()	13
5.2.2.3 getPrimeiraAcao()	13
5.2.2.4 removerAcao()	13
5.2.2.5 setId()	14
5.3 Referência da Classe Node	14
5.3.1 Construtores e Destrutores	14
5.3.1.1 Node()	14
5.3.1.2 ~Node()	14
5.3.2 Documentação das funções	15
5.3.2.1 getAnterior()	15

5.3.2.2	getData()	15
5.3.2.3	getProx()	15
5.3.2.4	setAnterior()	15
5.3.2.5	setData()	15
5.3.2.6	setProx()	15
5.4	Referência da Estrutura RankItem	16
5.4.1	Atributos	16
5.4.1.1	id	16
5.4.1.2	index	16
5.4.1.3	pontuacao	16
6	Arquivos	17
6.1	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Acao.hpp	17
6.2	Acao.hpp	17
6.3	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Cliente.hpp	18
6.4	Cliente.hpp	18
6.5	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Node.hpp	18
6.6	Node.hpp	19
6.7	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Acao.cpp	19
6.8	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Cliente.cpp	19
6.9	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/main.cpp	19
6.9.1	Funções	20
6.9.1.1	main()	20
6.9.1.2	merge()	23
6.9.1.3	mergeSort()	24
6.10	Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Node.cpp	24
	Índice Remissivo	25

Capítulo 1

Hierarquia de Diretórios

1.1 Diretórios

include	7
Acao.hpp	17
Cliente.hpp	18
Node.hpp	18
src	7
Acao.cpp	19
Cliente.cpp	19
main.cpp	19
Node.cpp	24
Trabalho1ED	7
include	7
Acao.hpp	17
Cliente.hpp	18
Node.hpp	18
src	7
Acao.cpp	19
Cliente.cpp	19
main.cpp	19
Node.cpp	24

Capítulo 2

Índice dos Componentes

2.1 Lista de Classes

Aqui estão as classes, estruturas, uniões e interfaces e suas respectivas descrições:

Acao	9
Cliente	12
Node	14
RankItem	16

Capítulo 3

Índice dos Arquivos

3.1 Lista de Arquivos

Esta é a lista de todos os arquivos e suas respectivas descrições:

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/ Acao.hpp	17
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/ Cliente.hpp	18
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/ Node.hpp	18
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/ Acao.cpp	19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/ Cliente.cpp	19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/ main.cpp	19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/ Node.cpp	24

Capítulo 4

Documentação do Diretório

4.1 Referência do diretório

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include

Arquivos

- arquivo [Acao.hpp](#)
- arquivo [Cliente.hpp](#)
- arquivo [Node.hpp](#)

4.2 Referência do diretório

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src

Arquivos

- arquivo [Acao.cpp](#)
- arquivo [Cliente.cpp](#)
- arquivo [main.cpp](#)
- arquivo [Node.cpp](#)

4.3 Referência do diretório

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED

Diretórios

- diretório [include](#)
- diretório [src](#)

Capítulo 5

Classes

5.1 Referência da Classe Acao

```
#include <Acao.hpp>
```

Membros Públicos

- `Acao` (int `_id`, `Node` *`primeiraCotacao`, `Node` *`ultimacotacao`)
- `~Acao` ()
- int `getId` () const
- void * `getUltimaCotacao` () const
- void `adicionarCotacao` (double `valor_cotacao`)
- double `calcularRetornoElementar` (int `qtd_cotacoes_consideradas`)
- double `calcularRet` (int `qtd_cotacoes_consideradas`)
- double `calcularAvgRet` (int `qtd_cotacoes_consideradas`)
- double `calcularStab` (int `qtd_cotacoes_consideradas`)
- double `calcularCons` (int `qtd_cotacoes_consideradas`)

5.1.1 Construtores e Destrutores

5.1.1.1 Acao()

```
Acao::Acao (  
    int _id,  
    Node * primeiraCotacao,  
    Node * ultimacotacao)  
  
00003                                     {  
00004     _id = id;  
00005     _primeiraCotacao = primeiraCotacao;  
00006     _ultimacotacao = ultimacotacao;  
00007     _numcotacoes = 0;  
00008 }
```

5.1.1.2 ~Acao()

```
Acao::~Acao () [default]
```

5.1.2 Documentação das funções

5.1.2.1 adicionarCotacao()

```
void Acao::adicionarCotacao (
    double valor_cotacao)
{
00039
00040     double *val_cotacao = new double();
00041     *val_cotacao = std::abs(valor_cotacao);
00042
00043     Node *cotacao = new Node(val_cotacao, nullptr, _ultimacotacao);
00044     if(_numcotacoes >= 1){
00045         _ultimacotacao->setProx(cotacao);
00046     }
00047     _ultimacotacao = cotacao;
00048     if(_primeiraCotacao == nullptr){
00049         _primeiraCotacao = cotacao;
00050     }
00051
00052     _numcotacoes++;
00053 }
00054 }
```

5.1.2.2 calcularAvgRet()

```
double Acao::calcularAvgRet (
    int qtd_cotacoes_consideradas)
{
00072
00073     if(qtd_cotacoes_consideradas > _numcotacoes || qtd_cotacoes_consideradas < 2){
00074         throw std::invalid_argument("Numero de cotacoes inválido");
00075     }
00076     double total_avgret = 0;
00077     Node *a = _ultimacotacao;
00078     for(int i = 0; i < qtd_cotacoes_consideradas-1; i++){
00079         if(a->getAnterior() == nullptr){
00080             throw std::runtime_error("A acao que deseja acessar é inválida ou não existe");
00081         }
00082         total_avgret += (*static_cast<double*>(a->getData())) /
00083             *static_cast<double*>(a->getAnterior()->getData()) - 1.0;
00084         a = a->getAnterior();
00085     }
00086     total_avgret = total_avgret * (1.0/(qtd_cotacoes_consideradas - 1.0));
00087     return total_avgret;
00088 }
```

5.1.2.3 calcularCons()

```
double Acao::calcularCons (
    int qtd_cotacoes_consideradas)
{
00107
00108     if(qtd_cotacoes_consideradas > _numcotacoes || qtd_cotacoes_consideradas < 2){
00109         throw std::invalid_argument("Numero de cotacoes inválido");
00110     }
00111     double cons=0, r_i;
00112     Node *a = _ultimacotacao;
00113     for(int i=0 ; i < qtd_cotacoes_consideradas-1; i++){
00114         if(a->getAnterior() == nullptr){
00115             throw std::runtime_error("A acao que deseja acessar é inválida ou não existe");
00116         }
00117         r_i = (*static_cast<double*>(a->getData())) /
00118             *static_cast<double*>(a->getAnterior()->getData()) - 1.0;
00119         if(r_i > 0){
00120             cons += 1;
00121         }
00122         a = a->getAnterior();
00123     }
00124     return (1.0/(qtd_cotacoes_consideradas-1.0)) * cons;
00125 }
```

5.1.2.4 calcularRet()

```
double Acao::calcularRet (
    int qtd_cotacoes_consideradas)
00056
00057     {
00058     if(qtd_cotacoes_consideradas > _numcotacoes || qtd_cotacoes_consideradas < 2){
00059         throw std::invalid_argument("Numero de cotacoes invalido");
00060     }
00061     Node *a = _ultimacotacao;
00062     for(int i = 0; i < qtd_cotacoes_consideradas - 1; i++){
00063         if(a->getAnterior() == nullptr) {
00064             throw std::runtime_error("A acao nao tem cotacoes suficientes.");
00065         }
00066         a = a->getAnterior();
00067     }
00068
00069     return (* (static_cast<double*>(_ultimacotacao->getData())) /
    *(static_cast<double*>(a->getData())))) - 1.0;
00070 }
```

5.1.2.5 calcularRetornoElementar()

```
double Acao::calcularRetornoElementar (
    int qtd_cotacoes_consideradas)
00023
00024     {
00025     Node *a = _ultimacotacao;
00026     if(posicao <= 0 || posicao > _numcotacoes-1){
00027         throw std::invalid_argument("Posicao inválida");
00028     }
00029     else if(_numcotacoes < 2){
00030         throw std::invalid_argument("Numero de cotacoes inválido");
00031     }
00032     double retorno_elementar = 0;
00033     for(int i = _numcotacoes-1; i > _numcotacoes - posicao && i>0; i--){
00034         a = a->getAnterior();
00035     }
00036     retorno_elementar = ((* (static_cast<double*>(a->getData())) /
    *(static_cast<double*>(a->getAnterior()->getData())))) - 1.0;
00037     return retorno_elementar;
00038 }
```

5.1.2.6 calcularStab()

```
double Acao::calcularStab (
    int qtd_cotacoes_consideradas)
00089
00090     {
00091     if(qtd_cotacoes_consideradas > _numcotacoes || qtd_cotacoes_consideradas < 2){
00092         throw std::invalid_argument("Numero de cotacoes inválido");
00093     }
00094     Node *a = _ultimacotacao;
00095     double total_avgret_r=0, avg_ret=calcularAvgRet(qtd_cotacoes_consideradas), vol;
00096     for(int i=0; i<qtd_cotacoes_consideradas-1; i++){
00097         if(a->getAnterior() == nullptr){
00098             throw std::runtime_error("A acao que deseja acessar é inválida ou não existe");
00099         }
00100         total_avgret_r += pow((((static_cast<double*>(a->getData()) /
    *(static_cast<double*>(a->getAnterior()->getData())) - 1.0) - avg_ret), 2);
00101         a = a->getAnterior();
00102     }
00103     vol = sqrt((1.0/(qtd_cotacoes_consideradas-1.0)) * total_avgret_r);
00104     return 1.0/(1.0 + vol);
00105 }
```

5.1.2.7 getId()

```
int Acao::getId () const
00013
00014     {
00015     return _id;
00016 }
```

5.1.2.8 getUltimaCotacao()

```
void * Acao::getUltimaCotacao () const
00017                                     {
00018     return _ultimacotacao;
00019 }
```

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Acao.hpp
- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Acao.cpp

5.2 Referência da Classe Cliente

```
#include <Cliente.hpp>
```

Membros Públicos

- [Cliente](#) (int id)
- [~Cliente](#) ()
- int [getId](#) () const
- [Node](#) * [getPrimeiraAcao](#) () const
- void [setId](#) (int id)
- void [adicionarAcao](#) ([Acao](#) *acao)
- void [removerAcao](#) (int id)

5.2.1 Construtores e Destrutores

5.2.1.1 Cliente()

```
Cliente::Cliente (
    int id)
00003                                     {
00004     _id = id;
00005     _primeiraAcao = nullptr;
00006     _ultimaAcao = nullptr;
00007     _numacoes = 0;
00008 }
```

5.2.1.2 ~Cliente()

```
Cliente::~~Cliente () [default]
```

5.2.2 Documentação das funções

5.2.2.1 adicionarAcao()

```
void Cliente::adicionarAcao (
    Acao * acao)

00029         {
00030     Node *nova_acao = new Node(acao, nullptr, _ultimaAcao);
00031     if(_primeiraAcao == nullptr){
00032         _primeiraAcao = nova_acao;
00033     }
00034     if(_numacoes >= 1){
00035         _ultimaAcao->setProx(nova_acao);
00036     }
00037     _ultimaAcao = nova_acao;
00038     _ultimaAcao->setProx(nullptr);
00039
00040
00041     _numacoes++;
00042 }
```

5.2.2.2 getId()

```
int Cliente::getId () const

00014     {
00015     return _id;
00016 }
```

5.2.2.3 getPrimeiraAcao()

```
Node * Cliente::getPrimeiraAcao () const

00018     {
00019     return _primeiraAcao;
00020 }
```

5.2.2.4 removerAcao()

```
void Cliente::removerAcao (
    int id)

00044     {
00045     Node *a = _primeiraAcao;
00046     while(a != nullptr){
00047         Acao* acao = static_cast<Acao*>(a->getData());
00048         if(acao->getId() == id){
00049             if(_numacoes == 1){
00050                 _ultimaAcao = nullptr;
00051                 _primeiraAcao = nullptr;
00052             }
00053
00054             else if(a->getAnterior() == nullptr){
00055                 a->getProx()->setAnterior(nullptr);
00056                 _primeiraAcao = a->getProx();
00057             }
00058             else if(a->getProx() == nullptr){
00059                 a->getAnterior()->setProx(nullptr);
00060                 _ultimaAcao = a->getAnterior();
00061             }
00062             else{
00063                 a->getAnterior()->setProx(a->getProx());
00064                 a->getProx()->setAnterior(a->getAnterior());
00065             }
00066             _numacoes--;
00067             delete a;
00068             return;
00069         }
00070         a = a->getProx();
00071     }
00072 }
```

5.2.2.5 setId()

```
void Cliente::setId (
    int id)
00023
00024     _id = id;
00025 }
```

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/[Cliente.hpp](#)
- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/[Cliente.cpp](#)

5.3 Referência da Classe Node

```
#include <Node.hpp>
```

Membros Públicos

- [Node](#) (void *data, [Node](#) *prox, [Node](#) *anterior)
- [~Node](#) ()
- void * [getData](#) () const
- [Node](#) * [getProx](#) () const
- [Node](#) * [getAnterior](#) () const
- void [setData](#) (void *data)
- void [setProx](#) ([Node](#) *prox)
- void [setAnterior](#) ([Node](#) *anterior)

5.3.1 Construtores e Destrutores

5.3.1.1 Node()

```
Node::Node (
    void * data,
    Node * prox,
    Node * anterior)
00003
00004     _data = data;
00005     _prox = prox;
00006     _anterior = anterior;
00007 }
```

5.3.1.2 ~Node()

```
Node::~Node () [default]
```

5.3.2 Documentação das funções

5.3.2.1 getAnterior()

```
Node * Node::getAnterior () const
00020     {
00021     return _anterior;
00022 }
```

5.3.2.2 getData()

```
void * Node::getData () const
00012     {
00013     return _data;
00014 }
```

5.3.2.3 getProx()

```
Node * Node::getProx () const
00016     {
00017     return _prox;
00018 }
```

5.3.2.4 setAnterior()

```
void Node::setAnterior (
    Node * anterior)
00034     {
00035     _anterior = anterior;
00036 }
```

5.3.2.5 setData()

```
void Node::setData (
    void * data)
00026     {
00027     _data = data;
00028 }
```

5.3.2.6 setProx()

```
void Node::setProx (
    Node * prox)
00030     {
00031     _prox = prox;
00032 }
```

A documentação para essa classe foi gerada a partir dos seguintes arquivos:

- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/[Node.hpp](#)
- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/[Node.cpp](#)

5.4 Referência da Estrutura RankItem

Atributos Públicos

- int [index](#)
- double [pontuacao](#)
- int [id](#)

5.4.1 Atributos

5.4.1.1 id

```
int RankItem::id
```

5.4.1.2 index

```
int RankItem::index
```

5.4.1.3 pontuacao

```
double RankItem::pontuacao
```

A documentação para essa estrutura foi gerada a partir do seguinte arquivo:

- C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/[main.cpp](#)

Capítulo 6

Arquivos

6.1 Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Acao.hpp

```
#include "Node.hpp"
#include <stdexcept>
#include <cmath>
#include <math.h>
```

Componentes

- class [Acao](#)

6.2 Acao.hpp

[Ir para a documentação desse arquivo.](#)

```
00001 #ifndef ACAO_HPP
00002 #define ACAO_HPP
00003 #include "Node.hpp"
00004 #include <stdexcept>
00005 #include <cmath>
00006 #include <math.h>
00007
00008 class Acao
00009 {
00010 private:
00011     int _id, _numcotacoes;
00012     Node *_primeiraCotacao;
00013     Node *_ultimacotacao;
00014
00015 public:
00016     Acao(int _id, Node *primeiraCotacao, Node *ultimacotacao);
00017     ~Acao();
00018
00019     //getters
00020     int getId() const;
00021     void* getUltimaCotacao() const;
00022
00023     //manipulacao de cotacoes
00024     void adicionarCotacao(double valor_cotacao);
00025
00026     //calcula das metricas
00027     double calcularRetornoElementar(int qtd_cotacoes_consideradas);
00028     double calcularRet(int qtd_cotacoes_consideradas);
00029     double calcularAvgRet(int qtd_cotacoes_consideradas);
00030     double calcularStab(int qtd_cotacoes_consideradas);
00031     double calcularCons(int qtd_cotacoes_consideradas);
00032 };
00033
00034 #endif
```

6.3 Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/↵ Trabalho1ED/include/Cliente.hpp

```
#include "Acao.hpp"  
#include "Node.hpp"
```

Componentes

- class [Cliente](#)

6.4 Cliente.hpp

[Ir para a documentação desse arquivo.](#)

```
00001 #ifndef CLIENTE_HPP  
00002 #define CLIENTE_HPP  
00003 #include "Acao.hpp"  
00004 #include "Node.hpp"  
00005  
00006 class Cliente  
00007 {  
00008 private:  
00009     int _id, _numacoes;  
00010     Node *_primeiraAcao;  
00011     Node *_ultimaAcao;  
00012  
00013 public:  
00014     Cliente(int id);  
00015     ~Cliente();  
00016  
00017     //getters  
00018     int getId() const;  
00019     Node* getPrimeiraAcao() const;  
00020  
00021     //setters  
00022     void setId(int id);  
00023  
00024     //manipulacao de acoes  
00025     void adicionarAcao(Acao *acao);  
00026     void removerAcao(int id);  
00027  
00028 };  
00029  
00030  
00031 #endif
```

6.5 Referência do Arquivo C:/Users/luigu/OneDrive/Documentos/↵ Trabalho1ED/include/Node.hpp

```
#include "Node.hpp"
```

Componentes

- class [Node](#)

6.6 Node.hpp

[Ir para a documentação desse arquivo.](#)

```
00001 #ifndef Node_HPP
00002 #define Node_HPP
00003 #include "Node.hpp"
00004
00005 class Node
00006 {
00007 private:
00008     void *_data;
00009     Node *_prox;
00010     Node *_anterior;
00011 public:
00012     Node(void *data, Node *prox, Node *anterior);
00013     ~Node();
00014
00015     //getters
00016     void *getData() const;
00017     Node *getProx() const;
00018     Node *getAnterior() const;
00019
00020     //setters
00021     void setData(void *data);
00022     void setProx(Node* prox);
00023     void setAnterior(Node* anterior);
00024 };
00025
00026 #endif
```

6.7 Referência do Arquivo

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Acao.cpp

```
#include "../include/Acao.hpp"
```

6.8 Referência do Arquivo

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Cliente.cpp

```
#include "../include/Cliente.hpp"
```

6.9 Referência do Arquivo

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/main.cpp

```
#include <iostream>
#include <string>
#include <sstream>
#include <fstream>
#include <stdexcept>
#include <iomanip>
#include "../include/Acao.hpp"
#include "../include/Cliente.hpp"
#include "../include/Node.hpp"
```

Componentes

- struct [RankItem](#)

Funções

- void [merge](#) ([RankItem](#) arr[], int const inicio, int const meio, int const fim)
- void [mergeSort](#) ([RankItem](#) arr[], int const inicio, int const fim)
- int [main](#) ()

6.9.1 Funções

6.9.1.1 main()

```
int main ()
00070 {
00071     int lineCounter = 0;
00072     char funcao;
00073
00074     Node acoes = Node(nullptr, nullptr, nullptr);
00075     Node *ultima_acao_cadastrada = &acoes;
00076
00077     Node clientes = Node(nullptr, nullptr, nullptr);
00078     Node *ultimo_cliente_cadastrado = &clientes;
00079
00080     int numcotacoes; // numero de cotacoes que serao consideradas nas metricas
00081     std::string param;
00082
00083     std::ifstream file("example.txt");
00084     std::string line;
00085
00086
00087     while (std::getline(std::cin, line)){
00088         if (line.empty())
00089             continue;
00090         lineCounter++;
00091         std::stringstream ss(line);
00092         ss >> funcao;
00093
00094         switch (funcao){
00095
00096             case 77:{ // M
00097                 ss >> numcotacoes;
00098                 break;
00099             }
00100
00101             //-----
00102             case 65:{ // A
00103                 double id_acao;
00104                 ss >> id_acao;
00105                 Acao *acao = new Acao(id_acao, nullptr, nullptr);
00106                 Node *nova_acao = new Node(acao, nullptr, ultima_acao_cadastrada);
00107                 ultima_acao_cadastrada->setProx(nova_acao);
00108                 ultima_acao_cadastrada = nova_acao;
00109                 break;
00110             }
00111             //-----
00112
00113             case 85:{ // U
00114                 int id_novo_cliente;
00115                 ss >> id_novo_cliente;
00116                 Cliente *cliente = new Cliente(id_novo_cliente);
00117                 Node *novoCliente = new Node(cliente, nullptr, ultimo_cliente_cadastrado);
00118                 ultimo_cliente_cadastrado->setProx(novoCliente);
00119                 ultimo_cliente_cadastrado = novoCliente;
00120                 break;
00121             }
00122
00123             //-----
00124
```

```

00125         case 80: { // P
00126             Node *a = acoes.getProx();
00127             int id_procurado;
00128             double cotacao;
00129             ss >> id_procurado;
00130             ss >> cotacao;
00131             while (a != nullptr) {
00132                 Acao *acao = static_cast<Acao *>(a->getData());
00133                 if (acao->getId() == id_procurado) {
00134                     acao->adicionarCotacao(cotacao);
00135                     break;
00136                 }
00137                 a = a->getProx();
00138             }
00139             break;
00140         }
00141     }
00142
00143     //-----
00144     case 66: { // B
00145         Node *a = acoes.getProx();
00146         Node *b = clientes.getProx();
00147         int id_cliente;
00148         int id_acao;
00149         ss >> id_cliente;
00150         ss >> id_acao;
00151
00152         while (a != nullptr) {
00153             if (static_cast<Acao *>(a->getData())->getId() == id_acao) {
00154                 break;
00155             }
00156             a = a->getProx();
00157         }
00158         if (a == nullptr) {
00159             throw std::invalid_argument("Comando inválido na linha " +
std::to_string(lineCounter));
00160         }
00161
00162         while (b != nullptr) {
00163             if (static_cast<Cliente *>(b->getData())->getId() == id_cliente) {
00164                 static_cast<Cliente *>(b->getData())->adicionarAcao(static_cast<Acao
*>(a->getData()));
00165                 break;
00166             }
00167             b = b->getProx();
00168         }
00169         break;
00170     }
00171
00172     //-----
00173     case 86: { // V
00174         Node *a = clientes.getProx();
00175         int id_cliente, id_acao;
00176         ss >> id_cliente;
00177         ss >> id_acao;
00178
00179         while (a != nullptr) {
00180             if (static_cast<Cliente *>(a->getData())->getId() == id_cliente) {
00181                 static_cast<Cliente *>(a->getData())->removerAcao(id_acao);
00182                 break;
00183             }
00184             a = a->getProx();
00185         }
00186
00187         break;
00188     }
00189
00190     //-----
00191
00192     case 81: {
00193         int id_consulta, id_cliente, num_acoes_retornadas, qtd_metricas;
00194         ss >> id_consulta >> id_cliente >> num_acoes_retornadas >> qtd_metricas;
00195
00196         std::string *metricas_nome = new std::string[qtd_metricas];
00197         double *pesos = new double[qtd_metricas];
00198         for (int i = 0; i < qtd_metricas; i++) {
00199             ss >> metricas_nome[i] >> pesos[i];
00200         }
00201
00202         int qtd_acoes = 0;
00203         Node *m_node = acoes.getProx();
00204         while (m_node != nullptr) {
00205             qtd_acoes++;

```

```

00207         m_node = m_node->getProx();
00208     }
00209
00210     if (qtd_acoes == 0 || numcotacoes < 2){
00211         delete[] metricas_nome;
00212         delete[] pesos;
00213         break;
00214     }
00215
00216     Acao **todas_acoes = new Acao *[qtd_acoes];
00217     double *pontuacao_global = new double[qtd_acoes];
00218
00219     m_node = acoes.getProx();
00220     for (int i = 0; i < qtd_acoes; i++){
00221         todas_acoes[i] = static_cast<Acao *>(m_node->getData());
00222         pontuacao_global[i] = 0.0;
00223         m_node = m_node->getProx();
00224     }
00225
00226     for (int j = 0; j < qtd_metricas; j++){
00227         RankItem *itens_global = new RankItem[qtd_acoes];
00228
00229         for (int i = 0; i < qtd_acoes; i++){
00230             itens_global[i].index = i;
00231             itens_global[i].id = todas_acoes[i]->getId();
00232             if (metricas_nome[j] == "RET"){
00233                 itens_global[i].pontuacao = todas_acoes[i]->calcularRet(numcotacoes);
00234             }
00235             else if (metricas_nome[j] == "AVGRET"){
00236                 itens_global[i].pontuacao = todas_acoes[i]->calcularAvgRet(numcotacoes);
00237             }
00238             else if (metricas_nome[j] == "STAB"){
00239                 itens_global[i].pontuacao = todas_acoes[i]->calcularStab(numcotacoes);
00240             }
00241             else if (metricas_nome[j] == "CONS"){
00242                 itens_global[i].pontuacao = todas_acoes[i]->calcularCons(numcotacoes);
00243             }
00244         }
00245
00246         mergeSort(itens_global, 0, qtd_acoes - 1);
00247
00248         for (int i = 0; i < qtd_acoes; i++){
00249             int original_idx = itens_global[i].index;
00250             int pontos = qtd_acoes - i;
00251             pontuacao_global[original_idx] += (pontos * pesos[j]);
00252         }
00253
00254         delete[] itens_global;
00255     }
00256
00257     Node *c_node = clientes.getProx();
00258     Cliente *cliente = nullptr;
00259     while (c_node != nullptr){
00260         if (static_cast<Cliente *>(c_node->getData())->getId() == id_cliente){
00261             cliente = static_cast<Cliente *>(c_node->getData());
00262             break;
00263         }
00264         c_node = c_node->getProx();
00265     }
00266
00267     if (cliente){
00268         int num_acoes_carteira = 0;
00269         Node *acao_node = cliente->getPrimeiraAcao();
00270         while (acao_node != nullptr){
00271             num_acoes_carteira++;
00272             acao_node = acao_node->getProx();
00273         }
00274
00275         if (num_acoes_carteira > 0){
00276             RankItem *array_carteira = new RankItem[num_acoes_carteira];
00277             acao_node = cliente->getPrimeiraAcao();
00278             int idx = 0;
00279             while (acao_node != nullptr){
00280                 Acao *ac = static_cast<Acao *>(acao_node->getData());
00281                 array_carteira[idx].id = ac->getId();
00282                 for (int i = 0; i < qtd_acoes; i++){
00283                     if (todas_acoes[i]->getId() == ac->getId()){
00284                         array_carteira[idx].pontuacao = pontuacao_global[i];
00285                         break;
00286                     }
00287                 }
00288                 idx++;
00289                 acao_node = acao_node->getProx();
00290             }
00291
00292             int max_imprimir;
00293             if (num_acoes_retornadas < num_acoes_carteira) max_imprimir = num_acoes_retornadas;

```

```

00294         else max_imprimir = num_acoes_carteira;
00295
00296         std::cout << std::fixed << std::setprecision(2);
00297
00298         mergeSort(array_carteira, 0, num_acoes_carteira - 1);
00299
00300         for (int i = 0; i < max_imprimir; i++){
00301             std::cout << "R " << id_consulta << " M " << i << " " << array_carteira[i].id << " "
00302             << array_carteira[i].pontuacao << std::endl;
00303         }
00304         for (int i = 0; i < max_imprimir; i++){
00305             int idx_pior = num_acoes_carteira - 1 - i;
00306             std::cout << "R " << id_consulta << " P " << i << " " << array_carteira[idx_pior].id
00307             << " " << array_carteira[idx_pior].pontuacao << std::endl;
00308         }
00309         delete[] array_carteira;
00310     }
00311
00312     delete[] metricas_nome;
00313     delete[] pesos;
00314     delete[] todas_acoes;
00315     delete[] pontuacao_global;
00316
00317     break;
00318 }
00319
00320 default:
00321     throw std::invalid_argument("Comando inválido na linha " + std::to_string(lineCounter));
00322     break;
00323 }
00324 }
00325 file.close();
00326 }

```

6.9.1.2 merge()

```

void merge (
    RankItem arr[],
    int const inicio,
    int const meio,
    int const fim)
{
00017
00018     int const n1 = meio - inicio + 1;
00019     int const n2 = fim - meio;
00020
00021     RankItem* array_esquerdo = new RankItem[n1];
00022     RankItem* array_direito = new RankItem[n2];
00023
00024     for (int i = 0; i < n1; i++)
00025         array_esquerdo[i] = arr[inicio + i];
00026     for (int j = 0; j < n2; j++)
00027         array_direito[j] = arr[meio + 1 + j];
00028
00029     int i = 0, j = 0, k = inicio;
00030
00031     while (i < n1 && j < n2) {
00032         if (array_esquerdo[i].pontuacao > array_direito[j].pontuacao ||
00033             (array_esquerdo[i].pontuacao == array_direito[j].pontuacao && array_esquerdo[i].id <
00034              array_direito[j].id)) {
00035             arr[k] = array_esquerdo[i];
00036             i++;
00037         } else {
00038             arr[k] = array_direito[j];
00039             j++;
00040         }
00041         k++;
00042     }
00043     while (i < n1) {
00044         arr[k] = array_esquerdo[i];
00045         i++;
00046         k++;
00047     }
00048     while (j < n2) {
00049         arr[k] = array_direito[j];
00050         j++;
00051         k++;
00052     }
}

```

```
00053     }
00054
00055     delete[] array_esquerdo;
00056     delete[] array_direito;
00057 }
```

6.9.1.3 mergeSort()

```
void mergeSort (
    RankItem arr[],
    int const inicio,
    int const fim)
{
00059
00060     if (inicio >= fim)
00061         return;
00062
00063     int meio = inicio + (fim - inicio) / 2;
00064     mergeSort(arr, inicio, meio);
00065     mergeSort(arr, meio + 1, fim);
00066     merge(arr, inicio, meio, fim);
00067 }
```

6.10 Referência do Arquivo

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Node.cpp

```
#include "../include/Node.hpp"
```

Índice Remissivo

~Acao
 Acao, 9
~Cliente
 Cliente, 12
~Node
 Node, 14

Acao, 9
 ~Acao, 9
 Acao, 9
 adicionarCotacao, 10
 calcularAvgRet, 10
 calcularCons, 10
 calcularRet, 10
 calcularRetornoElementar, 11
 calcularStab, 11
 getId, 11
 getUltimaCotacao, 11
adicionarAcao
 Cliente, 13
adicionarCotacao
 Acao, 10

C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Acao.hpp, 17
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Cliente.hpp, 18
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include/Node.hpp, 18, 19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Acao.cpp, 19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Cliente.cpp, 19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/main.cpp, 19
C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src/Node.cpp, 24
calcularAvgRet
 Acao, 10
calcularCons
 Acao, 10
calcularRet
 Acao, 10
calcularRetornoElementar
 Acao, 11
calcularStab
 Acao, 11
Cliente, 12
 ~Cliente, 12
 adicionarAcao, 13
 Cliente, 12
 getId, 13
 getPrimeiraAcao, 13
 removerAcao, 13
 setId, 13

getAnterior
 Node, 15
getData
 Node, 15
getId
 Acao, 11
 Cliente, 13
getPrimeiraAcao
 Cliente, 13
getProx
 Node, 15
getUltimaCotacao
 Acao, 11

id
 RankItem, 16
index
 RankItem, 16
main.cpp, 20
main, 20
merge, 23
mergeSort, 24
main.cpp, 23
main.cpp, 24
main.cpp, 24
Node, 14
 ~Node, 14
 getAnterior, 15
 getData, 15
 getProx, 15
 Node, 14
 setAnterior, 15
 setData, 15
 setProx, 15

pontuacao
 RankItem, 16

RankItem, 16

- id, [16](#)
- index, [16](#)
- pontuacao, [16](#)
- Referência do diretório C:/Users/luigu/OneDrive/Documentos/Trabalho1ED,
[7](#)
- Referência do diretório C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/include,
[7](#)
- Referência do diretório C:/Users/luigu/OneDrive/Documentos/Trabalho1ED/src,
[7](#)
- removerAcao
 - Cliente, [13](#)
- setAnterior
 - Node, [15](#)
- setData
 - Node, [15](#)
- setId
 - Cliente, [13](#)
- setProx
 - Node, [15](#)